Computer Architecture
x86 Architecture

Tatpong Katanyukul

CoE, KKU

8 มีนาคม พ.ศ. 2559

Evolution (1)

Year	CPU	Register width	Address width	Key features
1978	8086	16	20	Extend 8080 to 16-bit
1982	80286	16	24	Allow protected mode
1985	80386	32	32	+ new addressing modes and operations
1989	486	32	32	+ only a few instructions
	Implementation: improve pipelining and add 8KiB L1 cache and FPU			
1992	Pentium (P5)	32	32	+ only a few instructions
	Implementation: do superscalar and separate code and data cache			
1995	Pentium Pro (P6)	32	36	+ only a few instructions
	Implementation: do micro-operation, allow 36-bit address (PAE), and add L2 cache (256 KiB on different die, but in the CPU package)			

Evolution (2)

Year	CPU	Register width	Address width	Key features
1997	+ MMX	Multi Media Extension: 57 instructions using floating point stack to accelerate multimedia and communication applications		
1999	Pentium III	+ Streaming SIMD Extension (SSE) SSE: 70 instructions and 8 new registers, targeted for digital signal and graphics processing		
2001	Pentium 4	+ SSE2 SSE2: 144 instructions allowing 64-bit floating point operation in parallel		
2003	AMD K8	64	64	based on AMD64 (a.k.a. x86-64, x64, Intel64)

Evolution (3)

Year	CPU	Key features
2004	+ SSE3	SSE3: 13 instructions to support complex arithmetic, graphics operations, video encoding, floating point conversion, and thread synchronization
2006	+ SSE 4	SSE4: 54 instructions targeted graphics and multimedia processing.
	Implementation: shift to multi-core scheme.	
2007	+ EPT (along with VT-x)	Virtual Machine Extension (VMX): ~ 10 instructions supporting hardware-assisted virtualization
2008	Intel announces Intel 5 series: moving from CPU-Northbridge-Southbridge-motherboard design to CPU-Platform-controller-hub design	

Evolution (4)

Year	CPU	Key features
2011	+ AVX (e.g., 2th gen. Core)	Advanced Vector Extensions: expand SSE registers from 128 to 256 bits, thereby redefining $\sim$ 250 instructions and adding 128 new instructions
2013	Haswell (e.g., 4th gen. Core)	+ new instructions (AVX2, Gather, BMI, ABM, and FMA) for high performance computing
	Implementation: + on-chip GPUs with eDRAM	
2014	Broadwell (e.g., 5th gen. Core)	+ new instructions for multimedia, cryptographic, and high performance computing and memory protection
2015	Skylake (e.g., 6th gen. Core)	+ new instructions MPX and SGX, to improve memory protection

x86 Basic Execution Environment

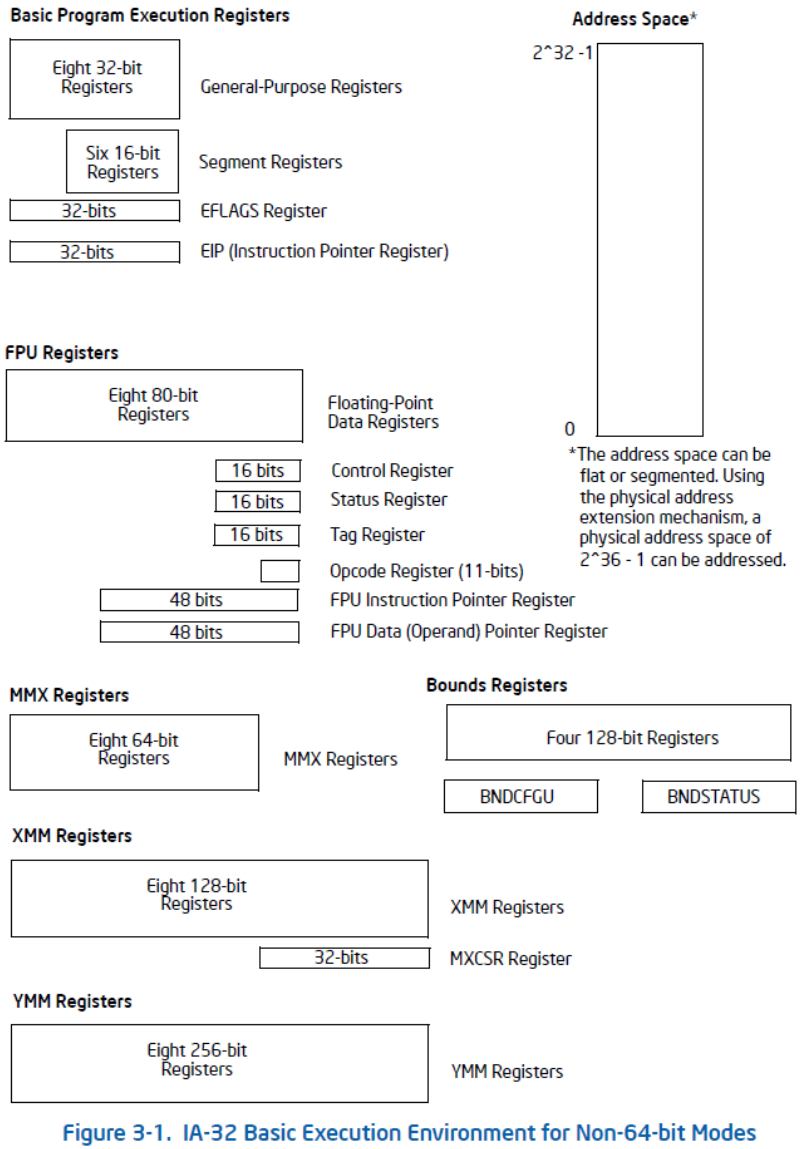
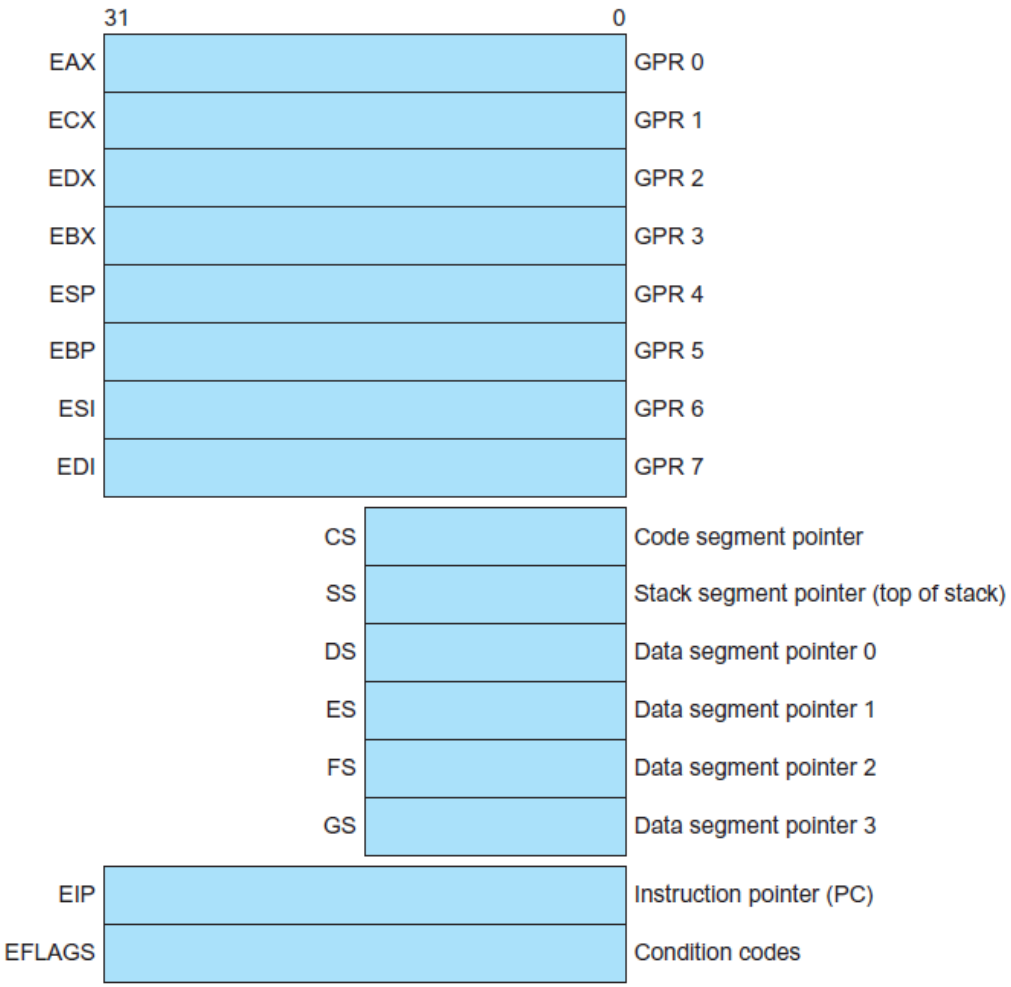


Figure 3-1. IA-32 Basic Execution Environment for Non-64-bit Modes

- Intel refers to a 32-bit version of x86 as IA-32.
- Bounds register, keeping lower and upper bounds of memory address.
  $\Rightarrow$ Providing a mechanism for efficient memory protection.
- MMX, XMM, and YMM are registers supporting SIMD.
- Pentium Pro (P6) has 36-bit address width.
  $\Rightarrow$ PAE allows full 36-bit address accessibility.
- Basic program execution environment has been seen since 80386.

x86 Basic Program Registers (1)



- Prefix 'E' indicates 32-bit version.
  Note, in 64-bit version, prefix 'r' is used.
- Registers, EAX, EBX, ..., EDI are general-purpose registers.
- Example

```
MOV EAX,   3A001500H
;   ≡ EAX ← 0x3A001500.
MOV ECX,   EAX
;   ≡ ECX ← EAX.
ADD EBX,   EAX
;   ≡ EBX ← EBX + EAX.
```

- Register EIP is equivalent to PC.

x86 Basic Program Registers (2)



- Register EFlags has information flags indicating conditions from the latest affected operations.
  $\Rightarrow$ such conditions are carry, parity, adjust, sign, etc.
- Segment registers, CS, SS, ..., GS, are to support segment addressing.

x86 Basic Program Registers (3)

- Arithmetic/logic instruction takes 2 operands.

  $\Rightarrow$ One operand acts as both source and destination, e.g., EBX in ADD EBX, EAX.

- A/L instruction can access memory.

```
ADD EAX, [1200H]                ; Absolute mode
ADD EAX, [EBX]                  ; Register indirect mode
ADD EAX, [EBX-110H]             ; Displacement mode
ADD EAX, [EBX*4 +0120H]         ; Scaled index with displacement mode
ADD EAX, [ECX + EBX*8 + 20H]    ; Base plus scaled index with displacement mode
ADD [230H], ECX                 ; Memory as source/destination
                                ; ≡ Memory[DS × 10H + 230H] ← [230H] + ECX.
```

- Not all registers are equal.

  $\Rightarrow$ There are restrictions on which registers can be used, e.g., ESP can't be used as an index.

Examples of x86 Basic Program Operations

`ADD EAX, 3450H`	;	EAX ← EAX + 0x3450.
`MOV [ECX], EBX`	;	Memory addressed by ECX gets value from EBX.
`CMP EAX, EBX`	;	Compare values of EAX and EBX (≡ doing EAX - EBX) and set condition codes in EFLAGS.
`JE LAB1`	;	≡ JZ, if zero flag in EFLAGS is 1, set EIP to the address corresponding to LAB1.
`JMP LAB2`	;	Set EIP to the address corresponding to LAB2.
`PUSH EAX`	;	Save EAX into the stack
`POP EAX`	;	Load value from the stack to EAX
`CALL func1`	;	Push EIP and set EIP to the address corresponding to func1.
`RET`	;	Pop a value in the stack to EIP.
`MOVSD`	;	Move 32-bit string from memory addressed by DS:ESI to location addressed by ES:EDI.

x86 Instruction Encoding (1)

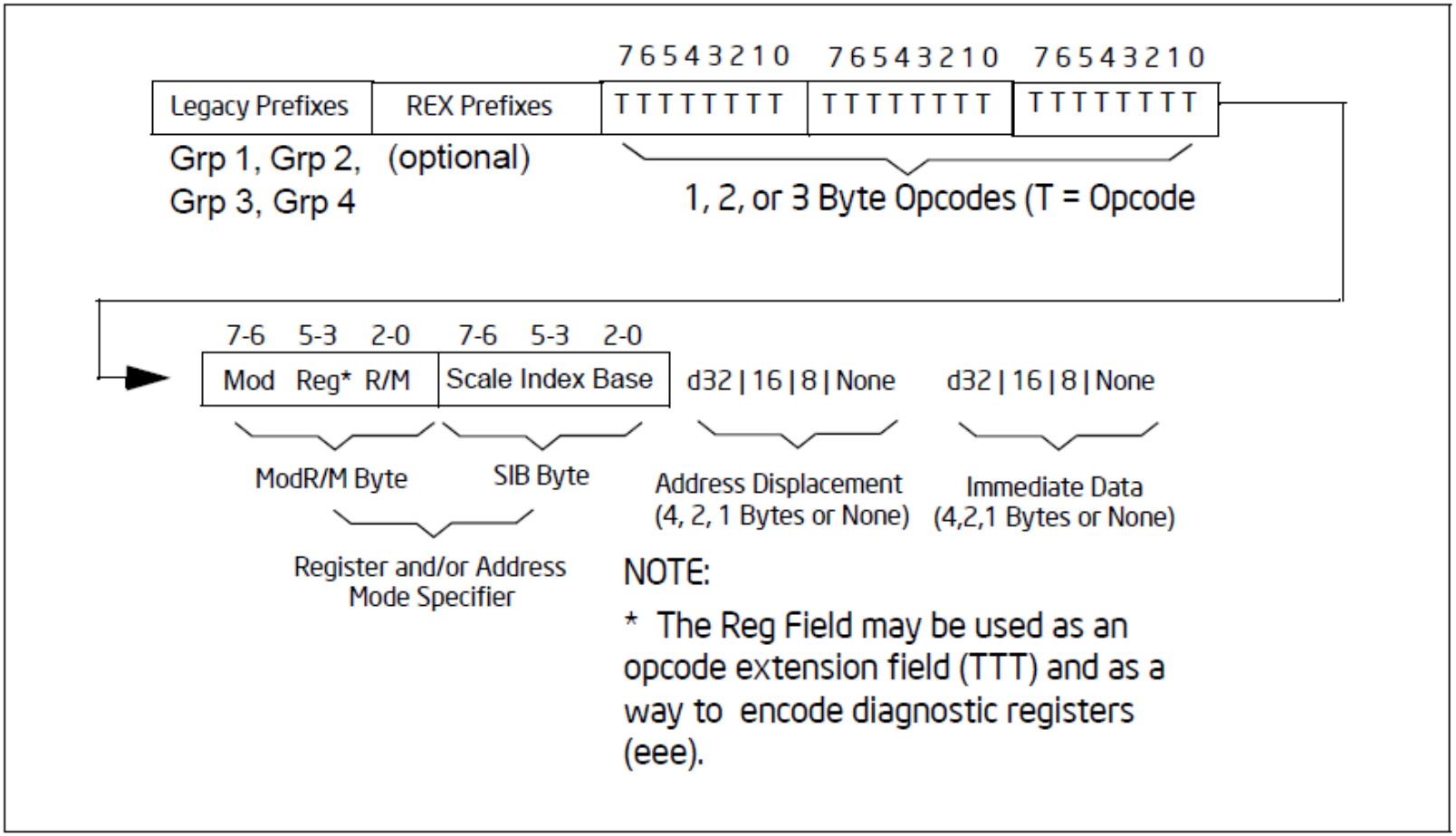


Figure B-1. General Machine Instruction Format

x86 Instruction Encoding (2)

- Opcode is 1- to 3-byte long.
  - Read 0x0F
    - followed by 0x38 or 0x3A $\Rightarrow$ 3-byte opcode,
    - otherwize $\Rightarrow$ 2-byte opcode.
  - Other than that, it is a 1-byte opcode.
- Each instruction can have from 0 to 4 prefixes:
  - Prefix modifies function of the instruction.
  - Each prefix takes 1 bytes.
- Instruction width (including prefixes) is $\sim$ 1 - 15 bytes.

Instruction Prefix

- Repeat/lock: 0xF0.

  lock memory $\Rightarrow$ have an exclusive use of all shared memory, until the instruction completes execution

- String manipulation:
  - 0xF3 $\Rightarrow$ repeat instruction the number of times specified by iteration count ECX.
  - 0xF2 $\Rightarrow$ repeat until termination on the value of ZF (in EFLAGS).

- Segment override:

Prefix:	0x2E	0x36	0x3E	0x26	0x64	0x65
Use:	CS	SS	DS	ES	FS	GS

  $\Rightarrow$ specify segment to use instead of a default.

- Operand override:

  0x66 $\Rightarrow$ changes size of data expected by default mode, e.g., 32-bit to 16-bit and vice versa.

- Address override:

  0x67 $\Rightarrow$ changes size of address expected by default mode, e.g., 32-bit to 16-bit and vice versa.

Encoding Example (1)

Example: Encode ADD ES:[EDX + 90003A00H], A4008245H.

See [Intel. Manual Vol. 2].

- First, determine the opcode.

mnemonic = ADD, operand 1 = memory addressing, operand 2 = immediate.

Table A-2. One-byte Opcode Map: (00H — F7H) *

	0	1	2	3	4	5	6	7
0	ADD Eb, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	rAX, Iz	PUSH ES^{i64}	POP ES^{i64}
1	ADC Eb, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	rAX, Iz	PUSH SS^{i64}	POP SS^{i64}
2	AND Eb, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	rAX, Iz	SEG=ES (Prefix)	DAA^{i64}
3	XOR Eb, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	rAX, Iz	SEG=SS (Prefix)	AAA^{i64}
4	INC^{i64} general register / REX^{o64} Prefixes eAX REX	eCX REX.B	eDX REX.X	eBX REX.XB	eSP REX.R	eBP REX.RB	eSI REX.RX	eDI REX.RXB
5	PUSH^{d64} general register rAX/r8	rCX/r9	rDX/r10	rBX/r11	rSP/r12	rBP/r13	rSI/r14	rDI/r15
6	PUSHA^{i64}/ PUSHAD^{i64}	POPA^{i64}/ POPAD^{i64}	BOUND^{i64} Gv, Ma	ARPL^{i64} Ew, Gw MOVSXD^{o64} Gv, Ev	SEG=FS (Prefix)	SEG=GS (Prefix)	Operand Size (Prefix)	Address Size (Prefix)
7	Jcc^{f64}, Jb - Short-displacement jump on condition O	NO	B/NAE/C	NB/AE/NC	Z/E	NZ/NE	BE/NA	NBE/A
8	Immediate Grp 1^{1A} Eb, Ib	Ev, Iz	Eb, Ib^{i64}	Ev, Ib	TEST Eb, Gb	Ev, Gv	XCHG Eb, Gb	Ev, Gv

Encoding Example (2)

A.2.1 Codes for Addressing Method

- E: A ModR/M byte follows the opcode and specifies the operand. The operand is either a general-purpose register or a memory address. If it is a memory address, the address is computed from a segment register and any of the following values: a base register, an index register, a scaling factor, a displacement.
- G: The reg field of the ModR/M byte selects a general register (for example, AX (000)).
- I: Immediate data: the operand value is encoded in subsequent bytes of the instruction.

A.2.2 Codes for Operand Type

- b Byte, regardless of operand-size attribute.
- v Word, doubleword or quadword (in 64-bit mode), depending on operand-size attribute.
- z Word for 16-bit operand-size or doubleword for 32 or 64-bit operand-size.

A.2.3 Register Codes
When an opcode requires a specific register as an operand, the register is identified by name (for example, AX, CL, or ESI).

rXX is used when 16, 32, or 64-bit sizes are possible.

$\Rightarrow$ ADD [EDX + 90003A00H], A4008245H needs ADD Ev, Iz $\Rightarrow$ opcode 0x81.

Excerpted from Section A.2

Encoding Example (3)

Some 1- and 2-byte opcodes point to group numbers (shaded entries in the opcode map table). Group numbers indicate that the instruction uses the reg/opcode bits in the ModR/M byte as an opcode extension (refer to Section A.4).

[Excerpted from Section A.2.]

A.4 OPCODE EXTENSIONS FOR ONE-BYTE AND TWO-BYTE OPCODES

Some 1-byte and 2-byte opcodes use bits 3-5 of the ModR/M byte (the nnn field in Figure A-1) as an extension of the opcode.

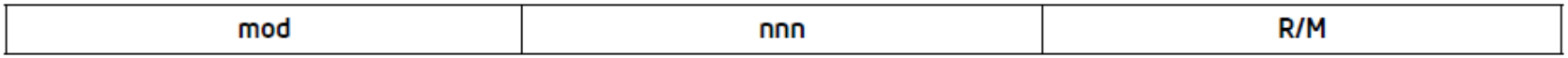


Figure A-1. ModR/M Byte nnn Field (Bits 5, 4, and 3)

Opcodes that have opcode extensions are indicated in Table A-6 and organized by group number. Group numbers (from 1 to 16, second column) provide a table entry point. The encoding for the r/m field for each instruction can be established using the third column of the table.

[Excerpted from Section A.4.]

Encoding Example (4)

Table A-6. Opcode Extensions for One- and Two-byte Opcodes by Group Number *

Opcode	Group	Mod 7,6	pfx	Encoding of Bits 5,4,3 of the ModR/M Byte (bits 2,1,0 in parenthesis)							
				000	001	010	011	100	101	110	111
80-83	1	mem, 11B		ADD	OR	ADC	SBB	AND	SUB	XOR	CMP
8F	1A	mem, 11B		POP							
C0,C1 reg, imm D0, D1 reg, 1 D2, D3 reg, CL	2	mem, 11B		ROL	ROR	RCL	RCR	SHL/SAL	SHR		SAR
F6, F7	3	mem, 11B		TEST Ib/Iz		NOT	NEG	MUL AL/rAX	IMUL AL/rAX	DIV AL/rAX	IDIV AL/rAX
FE	4	mem, 11B		INC Eb	DEC Eb						
FF	5	mem, 11B		INC Ev	DEC Ev	near CALL^{f64} Ev	far CALL Ep	near JMP^{f64} Ev	far JMP Mp	PUSH^{d64} Ev	
0F 00	6	mem, 11B		SLDT Rv/Mw	STR Rv/Mw	LLDT Ew	LTR Ew	VERR Ew	VERW Ew		
0F 01	7	mem		SGDT Ms	SIDT Ms	LGDT Ms	LIDT Ms	SMSW Mw/Rv		LMSW Ew	INVLPG Mb
		11B		VMCALL (001) VMLAUNCH (010) VMRESUME (011) VMXOFF (100)	MONITOR (000) MWAIT (001) CLAC (010) STAC (011)	XGETBV (000) XSETBV (001) VMFUNC (100) XEND (101) XTEST (110)					SWAPGS^{o64} (000) RDTSCP (001)
0F BA	8	mem, 11B						BT	BTS	BTR	BTC

⇒ ADD needs opcode extension ModR/M Bits [5,4,3] = 0b000.

Encoding Example (5)

Table 2-2. 32-Bit Addressing Forms with the ModR/M Byte

r8(/r) r16(/r) r32(/r) mm(/r) xmm(/r) (In decimal) /digit (Opcode) (In binary) REG =			AL AX EAX MM0 XMM0 0 000	CL CX ECX MM1 XMM1 1 001	DL DX EDX MM2 XMM2 2 010	BL BX EBX MM3 XMM3 3 011	AH SP ESP MM4 XMM4 4 100	CH BP EBP MM5 XMM5 5 101	DH SI ESI MM6 XMM6 6 110	BH DI EDI MM7 XMM7 7 111
Effective Address	**Mod**	**R/M**	**Value of ModR/M Byte (in Hexadecimal)**							
[EAX] [ECX] [EDX] [EBX] [--][--][1] disp32[2] [ESI] [EDI]	00	000 001 010 011 100 101 110 111	00 01 02 03 04 05 06 07	08 09 0A 0B 0C 0D 0E 0F	10 11 12 13 14 15 16 17	18 19 1A 1B 1C 1D 1E 1F	20 21 22 23 24 25 26 27	28 29 2A 2B 2C 2D 2E 2F	30 31 32 33 34 35 36 37	38 39 3A 3B 3C 3D 3E 3F
[EAX]+disp8[3] [ECX]+disp8 [EDX]+disp8 [EBX]+disp8 [--][--]+disp8 [EBP]+disp8 [ESI]+disp8 [EDI]+disp8	01	000 001 010 011 100 101 110 111	40 41 42 43 44 45 46 47	48 49 4A 4B 4C 4D 4E 4F	50 51 52 53 54 55 56 57	58 59 5A 5B 5C 5D 5E 5F	60 61 62 63 64 65 66 67	68 69 6A 6B 6C 6D 6E 6F	70 71 72 73 74 75 76 77	78 79 7A 7B 7C 7D 7E 7F
[EAX]+disp32 [ECX]+disp32 [EDX]+disp32 [EBX]+disp32 [--][--]+disp32 [EBP]+disp32 [ESI]+disp32 [EDI]+disp32	10	000 001 010 011 100 101 110 111	80 81 82 83 84 85 86 87	88 89 8A 8B 8C 8D 8E 8F	90 91 92 93 94 95 96 97	98 99 9A 9B 9C 9D 9E 9F	A0 A1 A2 A3 A4 A5 A6 A7	A8 A9 AA AB AC AD AE AF	B0 B1 B2 B3 B4 B5 B6 B7	B8 B9 BA BB BC BD BE BF
EAX/AX/AL/MM0/XMM0 ECX/CX/CL/MM/XMM1 EDX/DX/DL/MM2/XMM2 EBX/BX/BL/MM3/XMM3 ESP/SP/AH/MM4/XMM4 EBP/BP/CH/MM5/XMM5 ESI/SI/DH/MM6/XMM6 EDI/DI/BH/MM7/XMM7	11	000 001 010 011 100 101 110 111	C0 C1 C2 C3 C4 C5 C6 C7	C8 C9 CA CB CC CD CE CF	D0 D1 D2 D3 D4 D5 D6 D7	D8 D9 DA DB DC DD DE DF	E0 E1 E2 E3 E4 E5 E6 E7	E8 E9 EA EB EC ED EE EF	F0 F1 F2 F3 F4 F5 F6 F7	F8 F9 FA FB FC FD FE FF

$\Rightarrow$ [EDX + 90003A00H] needs [Mod = 0b10] and [R/M = 0b010].

Encoding Example (6)

ADD ES: [EDX + 90003A00H] , A4008245H

Options

- ADD Ev, Iz ⇒ Opcode = 0x81 and ModR/M[5-3] = 0b000.
- ES: ⇒ Opcode = 0x26.
- EDX + disp32 ⇒ ModR/M[7,6] = 0b10 and ModR/M[2-0] = 0b010.

Prefix	Opcode	ModR/M	SIB	Displacement	Immediate
0x26	0x81	0b10 000 010 = 0x82		0x90003A00	0xA4008245
Override default segment to ES	Table A.1	Table A.6 for bits [5-3] Table 2.2 for bits [7,6,2-0]	Not used		
1 byte	1 byte	1 byte	0 byte	4 bytes	4 bytes

∴ ADD ES:[EDX + 90003A00H], A4008245H ⇒ 0x26818290003A00A4008254.

Real Mode

Real mode (as the only mode in 8086)

- 20 address bits $\Rightarrow 2^{20} = 1,048,576$ addressable spaces = 1 MiB
- No restriction: any software can access any memory space.
- Real mode addressing
  - 8086 does it by: 4-bit-shifted segment + offset
  - (1) Specify segmentation into a 16-bit segment register, e.g., DS has 0x01E4.

    MOV AX, 01E4h

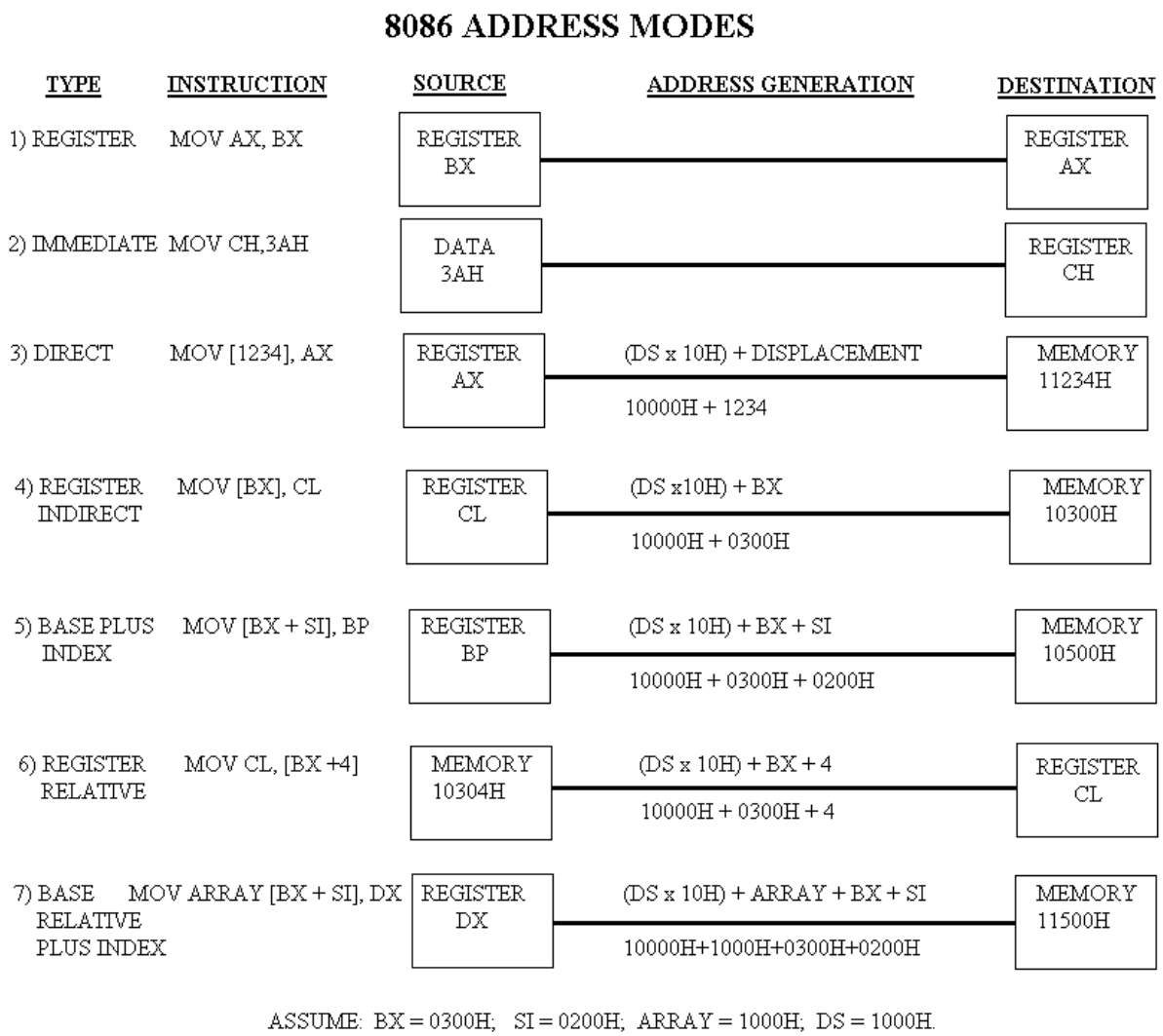
    MOV DS, AX
  - (2) Specify the offset, e.g., DX has 0x1623.

    MOV DX, 1623h
  - (3) This will refer to an address of 0x01E40 + 0x1623 = 0x03463.

    MOV BX, [DX]
  - Note: Given segmentation is fixed, offset alone can refer to 64 KiB.

    ($\equiv$ each segment has a size of 64KiB.)

Real Mode



(image src: http://www.electronics.dit.ie/staff/tscarff/8086_address_modes/8086_address_modes.htm)

Real Mode

- segment register is defaulted based on a function of memory access
  - Fetch an instruction: code segment register (CS).
  - Access stack, e.g., push, pop: stack segment register (SS).
  - Access data, e.g., mov: data segment register (DS).
  - Operate on strings, e.g., movs, cmps: extra segment register (ES).
- The instruction format allows an optional segment prefix byte to override the default:

```
MOV BX, [DX]        ;  use default DS for segment
MOV BX, ES:[DX]     ;  use ES for segment
```

- Note: an x86 processor starts in real mode.

Protected Mode

- x86 processor always starts up in a real mode for backward compatibility reason.
- introduced in i286.
  - i286 allows protected mode. Once in protected mode, no way to go back to real mode (unless restarting).
  - Intel fixed this in i386, which allowed system to go back to real mode.
- Protection
  - allows control of how programs can access different types of segments
  - facilitates limit checking (to ensure memory access within the limit of the segment)
  - maintains privilege levels or who has access to a segment.
  - controls access to privileged instructions.

Simplified Protected Mode Mechanism I

In protected mode, memory segments can be assigned protection.

- Remark: this is what hardware provides. It also depends on how to use it. (I.e. software part, specifically it's the OS job.)
- Provide a privilege level for code.
- Use Bit 0 (PE) of control register CR0 to indicate protected mode.

  $\Rightarrow$ Instruction MOV CR0, reg provides a mechanism to set/reset PE.
  E.g.,

```
MOV EAX, CR0
OR EAX, 1
MOV CR0, EAX
```

  $\Rightarrow$ Control register can be accessed only through a privilege instruction.

  $\Rightarrow$ Note: OS code must assign itself the highest privilege level (level 0) and other programs lower levels.

[See Intel Manual Vol 3, Chapter 5 Protection]

Simplified Protected Mode Mechanism II

In protected mode, memory segments can be assigned protection.

- Memory addressing is through a selector register and a descriptor table (either GDT or LDT) in addressing mode.
  - Selector register indicates descriptor table, an entry on the table, and a privilege level.
  - Descriptor table keeps information of memory segments: base address, limit, and attributes (e.g., privilege).
  - Every memory access is checked against the segment limit.
  - Any attempt to violate this protection causes a “General Protection” exception.

System Registers

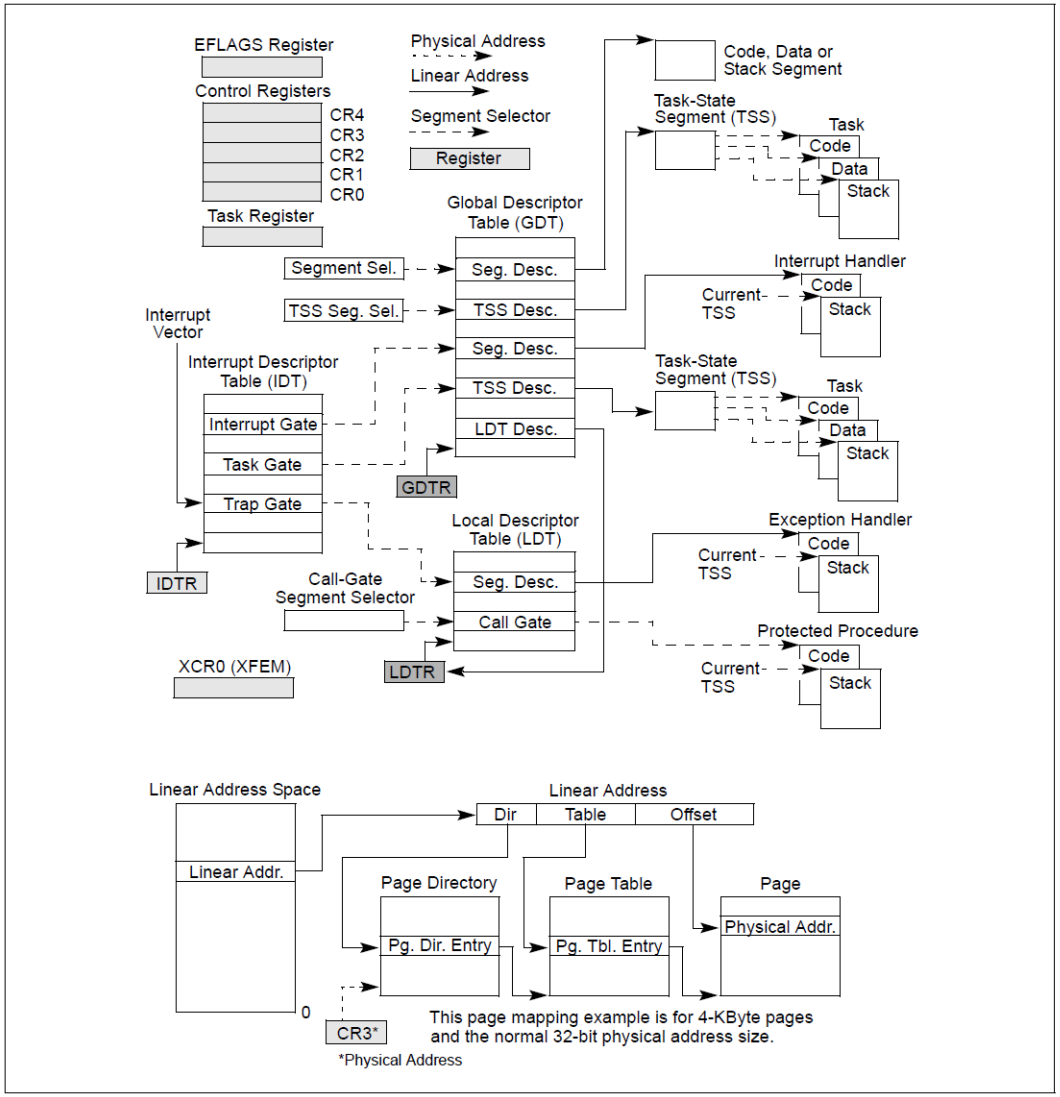


Figure 2-1. IA-32 System-Level Registers and Data Structures

[Intel. Manual Vol 3]

Control Registers

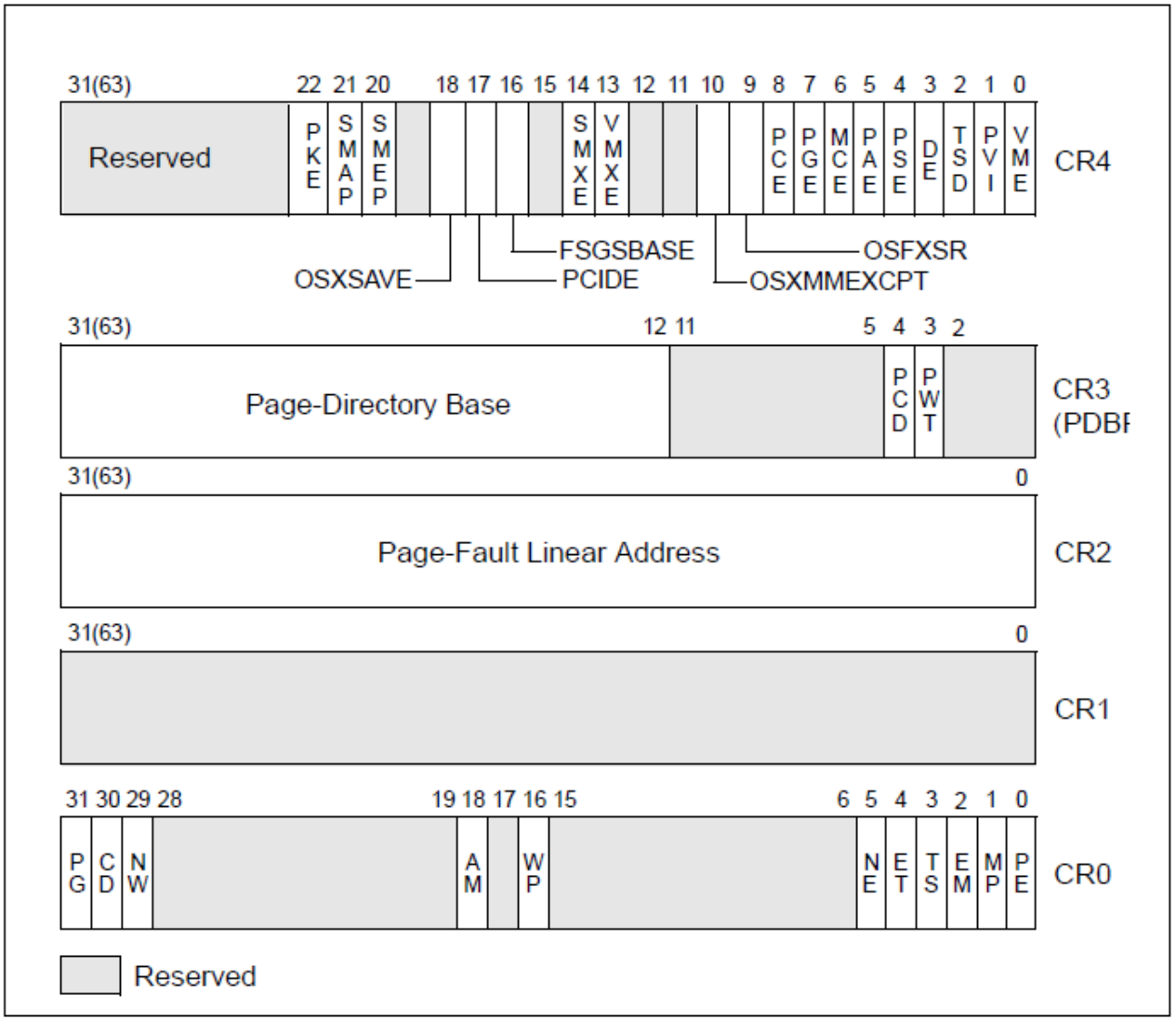


Figure 2-7. Control Registers

[Intel. Manual Vol 3]

Linear Addressing in Protected Mode

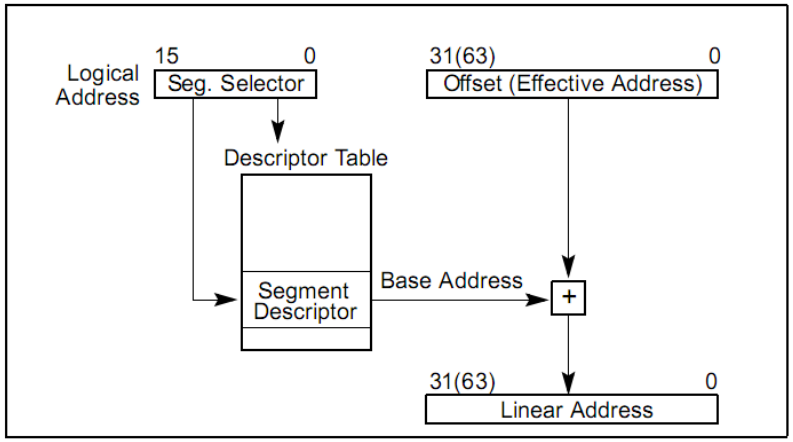


Figure 3-5. Logical Address to Linear Address Translation

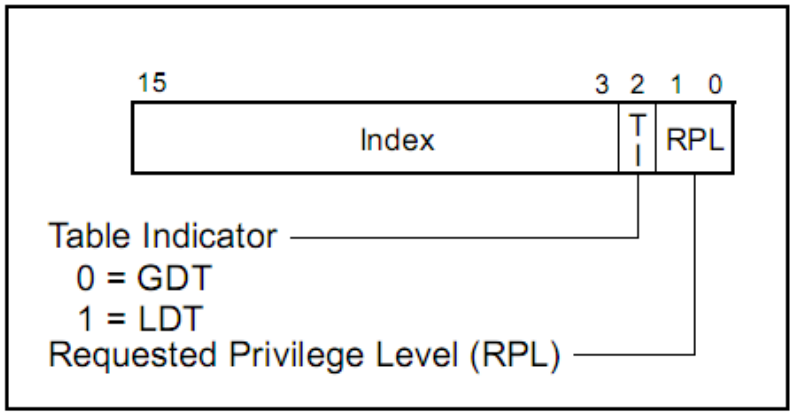


Figure 3-6. Segment Selector

Options

- 1. Without paging, linear address $\Rightarrow$ physical address (the linear address goes out on the processor's address bus.)
- 2. With paging, "the processor uses the page directory and page tables to translate the linear address into a physical address."

[Intel. Manual Vol 3]

Linear Addressing and Paging

- “Paging supports a "virtual memory" environment where a large linear address space is simulated with a small amount of physical memory (RAM and ROM) and some disk storage. When using paging, each segment is divided into pages (typically 4 KBytes each in size), which are stored either in physical memory or on the disk.”
- “When a program (or task) attempts to access an address location in the linear address space, the processor uses the page directory and page tables to translate the linear address into a physical address and then performs the requested operation (read or write) on the memory location. If the page being accessed is not currently in physical memory, the processor interrupts execution of the program (by generating a page-fault exception). The operating system or executive then reads the page into physical memory from the disk and continues executing the program.”

[Intel. Manual Vol 3]

Linear Address and Paging

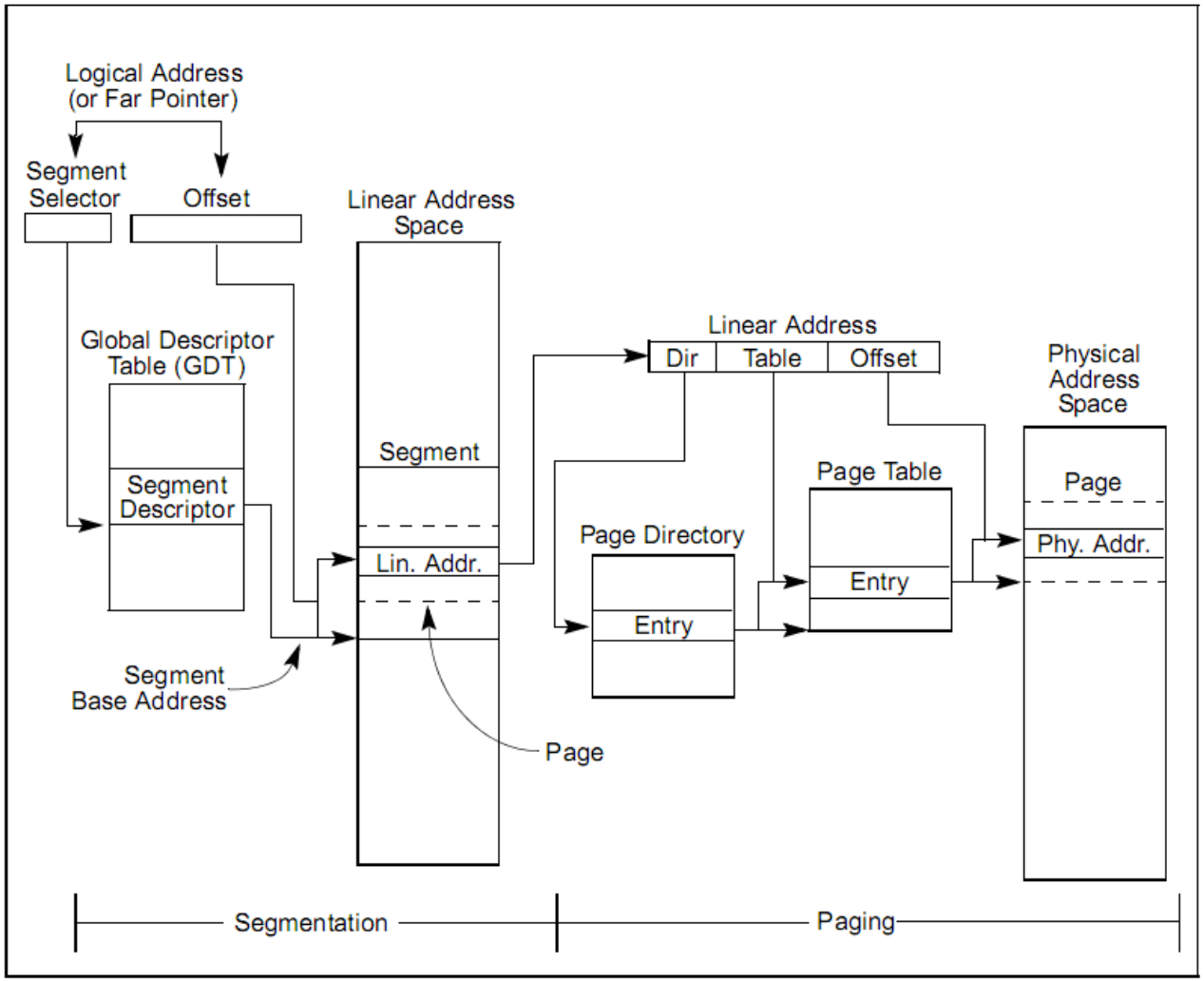


Figure 3-1. Segmentation and Paging

[Intel. Manual Vol 3]

Protection

- CR0.PE = 1 $\Rightarrow$ Protected mode
- The protection checks
  - Limit checks

    E.g., Access to any address $>$ the effective limit $\Rightarrow$ general-protection exception
  - Type checks

    E.g., "The CS register only can be loaded with a selector for a code segment."
  - Privilege level checks

    E.g., the currently executing program cannot access data segment when its Current Privilege Level (CPL, stored in bits 0 and 1 of segment registers CS and SS) is higher than Descriptor Privilege Level (DPL) of the data segment descriptor.
  - Restriction of instruction set, addressable domain, and procedure entry-points

    E.g., privilege instructions can be executed only when the CPL is 0 (most privileged)

Protection

5.2 FIELDS AND FLAGS USED FOR SEGMENT-LEVEL AND PAGE-LEVEL PROTECTION

The processor's protection mechanism uses the following fields and flags in the system data structures to control access to segments and pages:

- **Descriptor type (S) flag —** (Bit 12 in the second doubleword of a segment descriptor.) Determines if the segment descriptor is for a system segment or a code or data segment.
- **Type field** — (Bits 8 through 11 in the second doubleword of a segment descriptor.) Determines the type of code, data, or system segment.
- **Limit field** — (Bits 0 through 15 of the first doubleword and bits 16 through 19 of the second doubleword of a segment descriptor.) Determines the size of the segment, along with the G flag and E flag (for data segments).
- **G flag —** (Bit 23 in the second doubleword of a segment descriptor.) Determines the size of the segment, along with the limit field and E flag (for data segments).
- **E flag —** (Bit 10 in the second doubleword of a data-segment descriptor.) Determines the size of the segment, along with the limit field and G flag.
- **Descriptor privilege level (DPL) field** — (Bits 13 and 14 in the second doubleword of a segment descriptor.) Determines the privilege level of the segment.
- **Requested privilege level (RPL) field** — (Bits 0 and 1 of any segment selector.) Specifies the requested privilege level of a segment selector.
- **Current privilege level (CPL) field** — (Bits 0 and 1 of the CS segment register.) Indicates the privilege level of the currently executing program or procedure. The term current privilege level (CPL) refers to the setting of this field.
- **User/supervisor (U/S) flag** — (Bit 2 of paging-structure entries.) Determines the type of page: user or supervisor.
- **Read/write (R/W) flag** — (Bit 1 of paging-structure entries.) Determines the type of access allowed to a page: read-only or read/write.
- **Execute-disable (XD) flag** — (Bit 63 of certain paging-structure entries.) Determines the type of access allowed to a page: executable or not-executable.

[Intel. Manual Vol 3, Chapter 5]

Protection

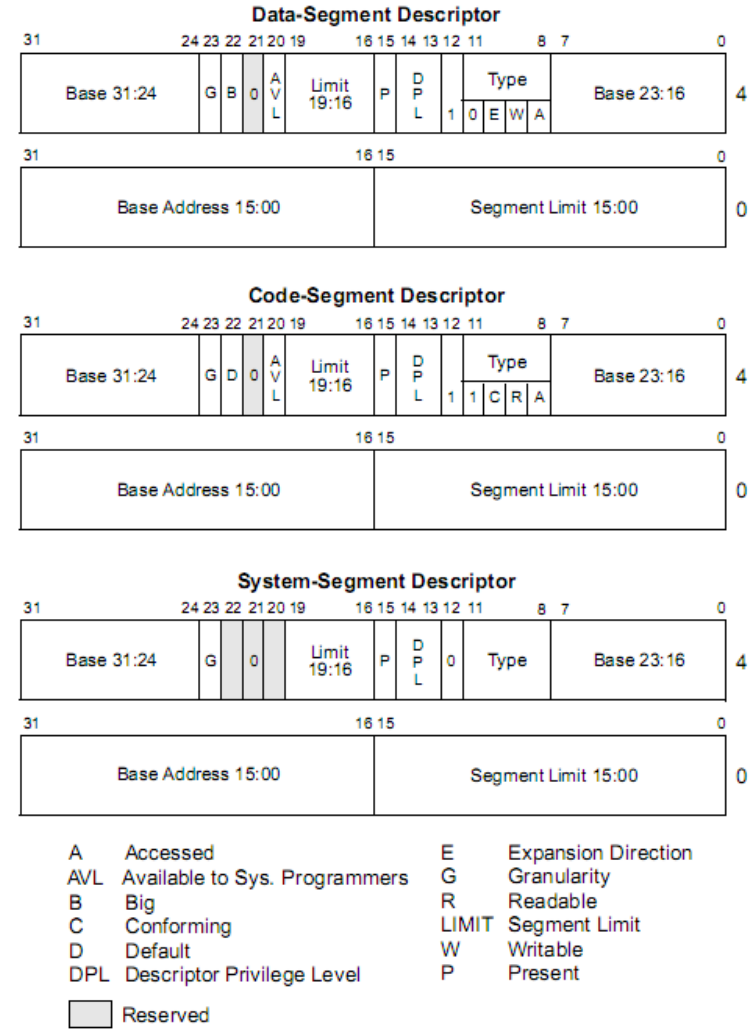


Figure 5-1. Descriptor Fields Used for Protection

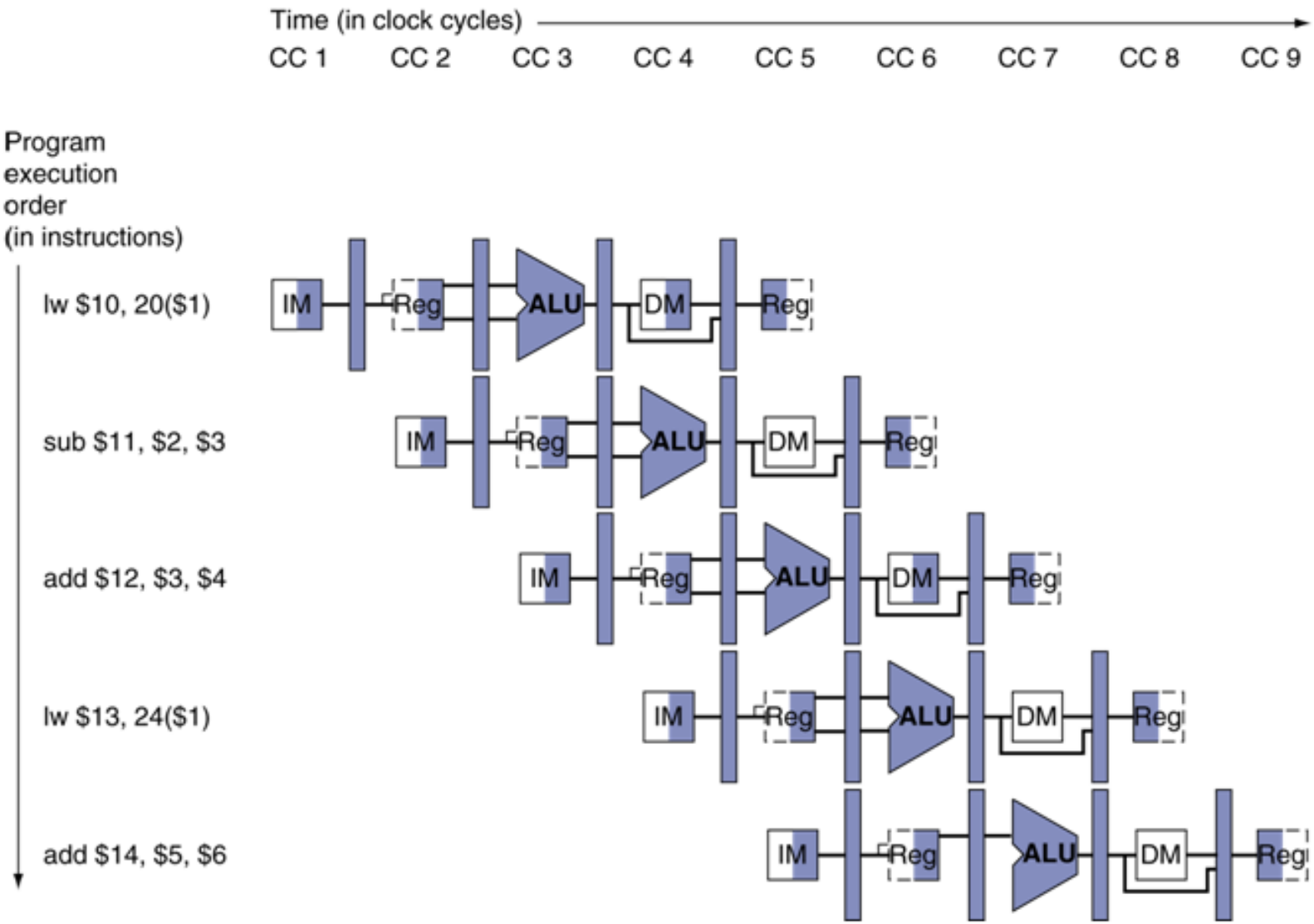
[Intel. Manual Vol 3, Chapter 5]

Parallelism

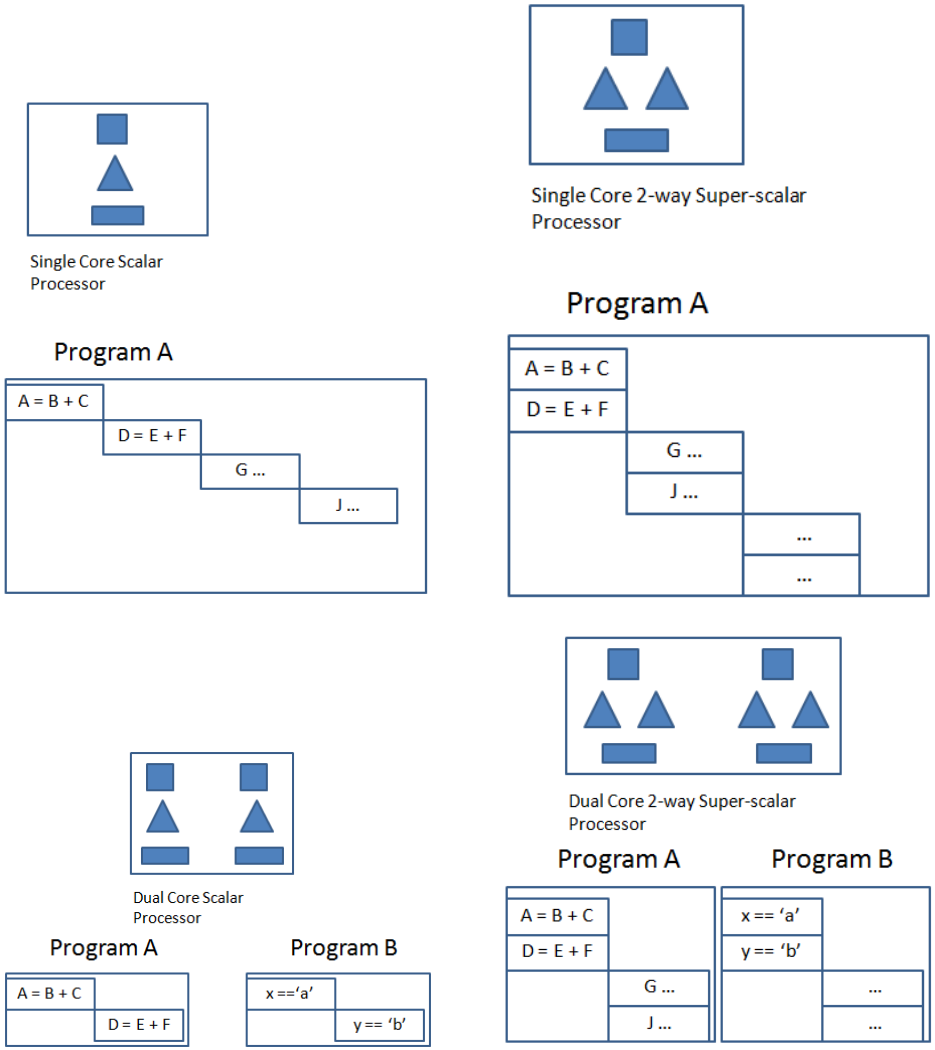
E.g.,

- Pipelining
- Multi-Core
- Superscalar
- Simultaneous Multi-Threading
- Single Instruction Multiple Data (SIMD)

Pipelining

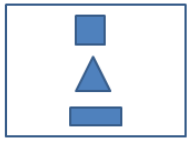

Time (in clock cycles)
CC 1
CC 2
CC 3
CC 4
CC 5
CC 6
CC 7
CC 8
CC 9
Program execution order (in instructions)
lw $10, 20($1)
sub $11, $2, $3
add $12, $3, $4
lw $13, 24($1)
add $14, $5, $6
IM
Reg
ALU
DM
Reg

Multi-core v.s. Superscalar (1)

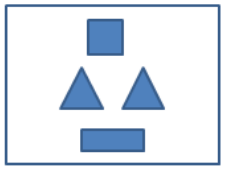


- Superscalar: "Each execution unit is not a separate processing unit (called "core") as in multi-core processors, but an execution resource within a single CPU such as an arithmetic logic unit, a bit shifter, or a multiplier." [wikipedia]

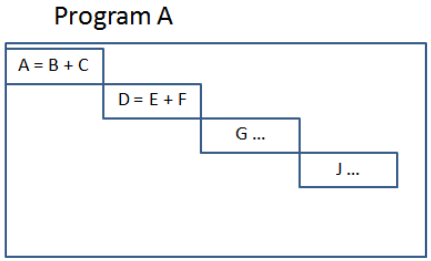
Multi-core v.s. Superscalar (2)

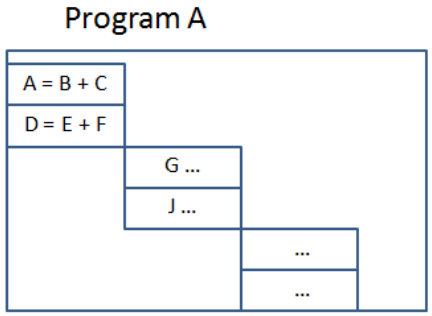


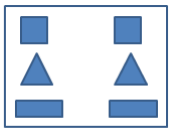
Single Core Scalar Processor



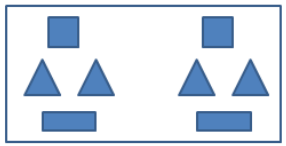
Single Core 2-way Super-scalar Processor



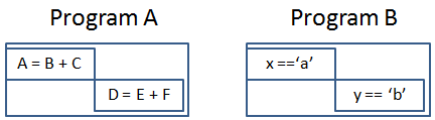


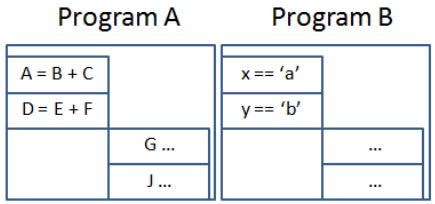


Dual Core Scalar Processor



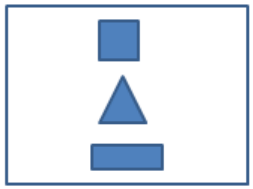
Dual Core 2-way Super-scalar Processor



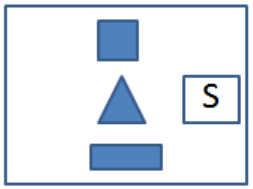


- E.g., Superscalar pipelining: fast prediction execution scheme simultaneously executes both branches and the throws away the “wrong” branch result.

Multi-Threading

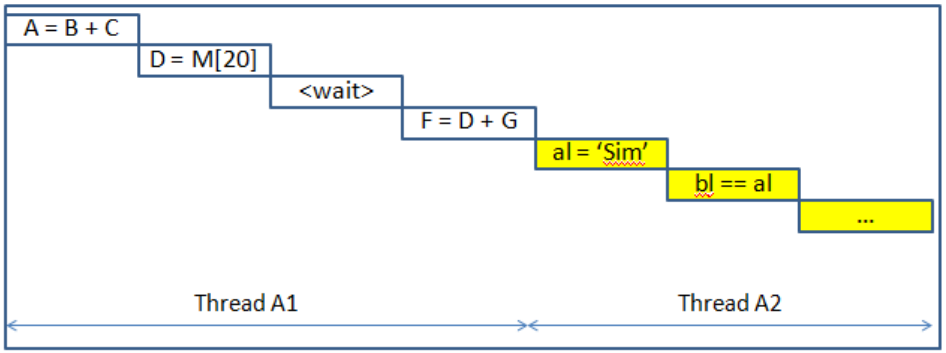


Single Core Scalar Processor
(no hardware thread support)

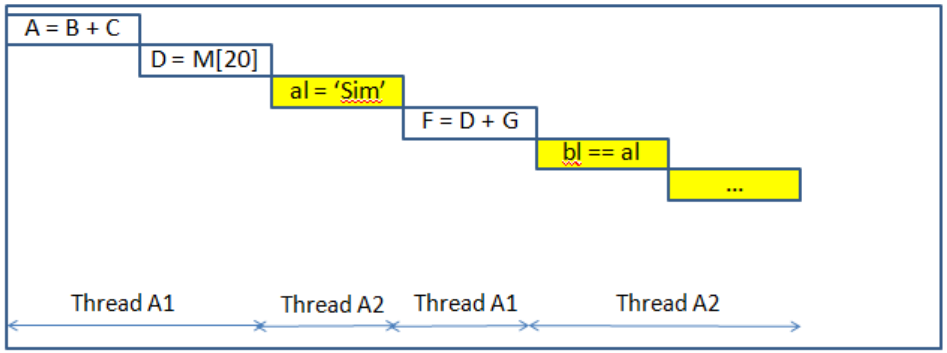


Single Core Scalar Processor
(with hardware thread support)

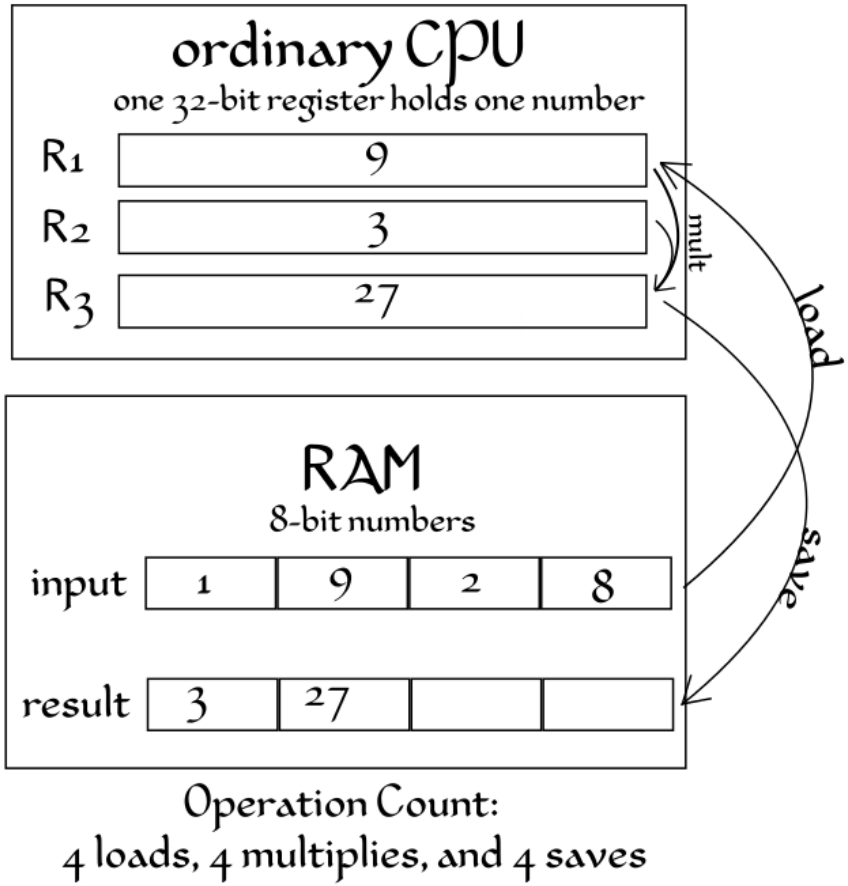
Multi-thread Program A (scheduled by OS)



Multi-thread Program A (scheduled by OS & HW)

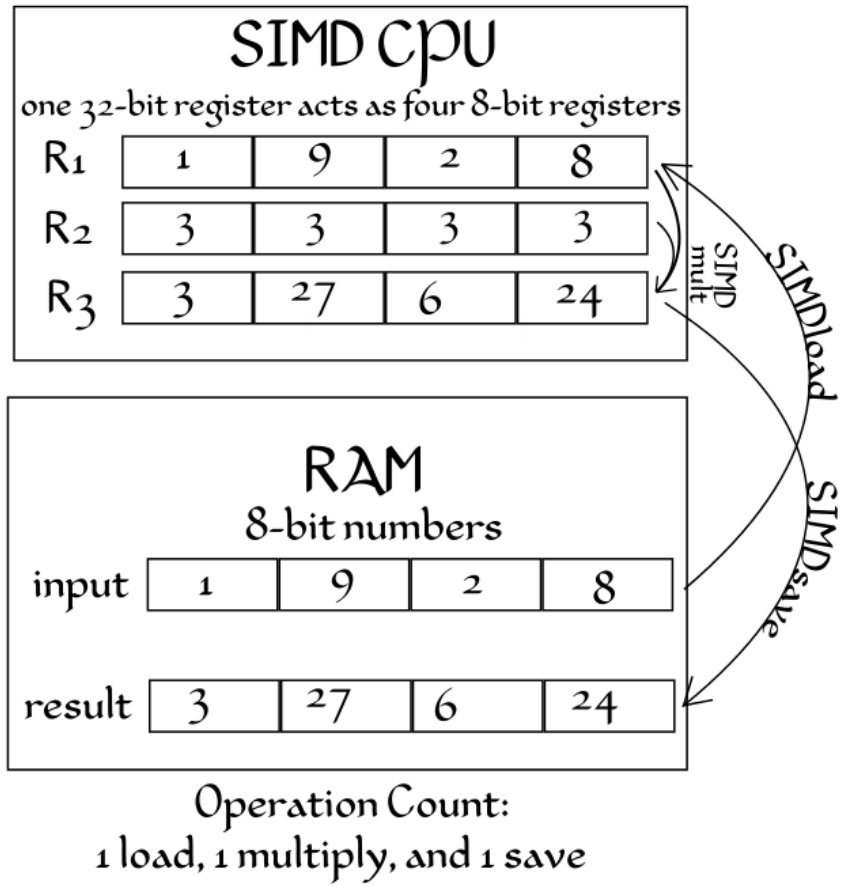


SIMD



src: Decora 2011

https://en.wikipedia.org/wiki/File:Non-SIMD_cpu_diagram1.svg



src: Decora 2011

https://en.wikipedia.org/wiki/File:SIMD_cpu_diagram1.svg

Closing note

- อ่าน P&H5e, 2.17.
- [Intel. Manual Vol 1] Intel 64 and IA-32 Architectures Software Developer's Manual Volume 1: Basic Architecture. Dec 2015.
- [Intel. Manual Vol 2] Intel 64 and IA-32 Architectures Software Developer's Manual Volume 2: Instruction Set Reference (A-Z). Dec 2015.
- [Intel. Manual Vol 3] Intel 64 and IA-32 Architectures Software Developer's Manual Volume 3: System Programming Guide. Dec 2015.

